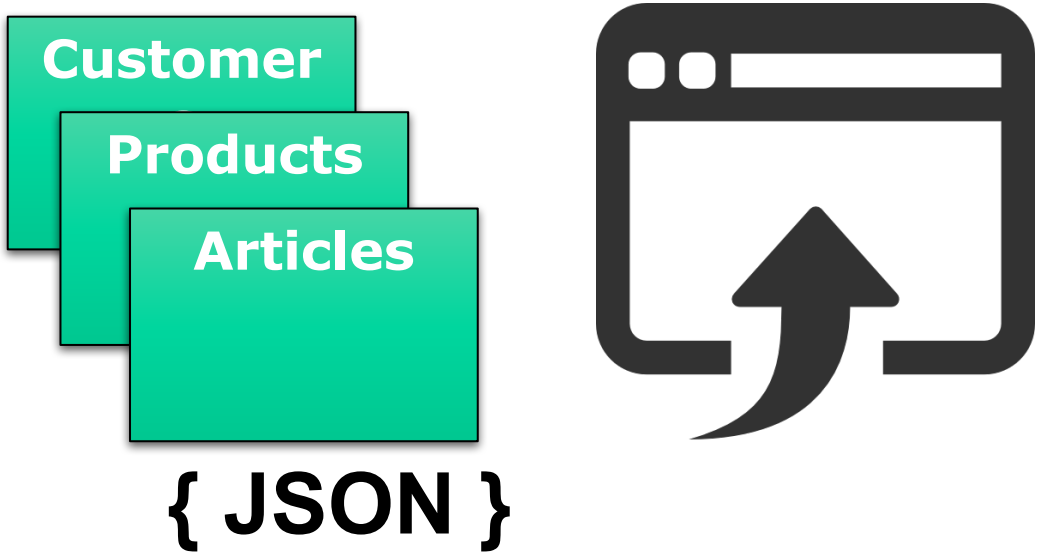
Angular 2
Module 2 - Databinding

Peter Kassenaar –
info@kassenaar.com

Wat is databinding

- Gegevens (data) tonen in de user interface
- Data afkomstig uit:
  - Controller / class
  - Database
  - User input
  - Andere systemen



Declaratieve syntaxis

- Nieuwe notatiewijzen in HTML-views/partials.
    1. Simple data binding
    2. Event binding
    3. One-way data binding
    4. Two-way data binding
- Angular 1:
    - Views zijn op zichzelf staande HTML-documenten. Krijgen via router in een app onderlinge samenhang
- Angular 2:
    - Views horen bij een bepaalde component.

1. Simple data binding syntax

Ongewijzigd ten opzichte van Angular 1. Dus nog steeds dubbele accolades:

```
<div>Stad: {{ city }}</div>

<div>Voornaam: {{ person.firstname }}</div>
```

Altijd: samenwerking met component/class

```
import {Component} from '@angular/core';
@Component({
   selector: 'hello-world',
   template: `<h1>Hello Angular 2</h1>
      <h2>Mijn naam is : {{ name }}</h2>
      <h2>Mijn favoriete stad is : {{ city }}</h2>
   `
})
export class AppComponent {
   name = 'Peter Kassenaar';
   city = 'Groningen'
}
```

Of: properties via constructor

```
export class AppComponent {
   name: string;
   city: string;

   constructor() {
      this.name = 'Peter Kassenaar';
      this.city = 'Groningen'
   }
}
```

Vaak: persoonlijke voorkeur, of coding style/ organization preferences

- BEST PRACTICE: In ngOnInit()`

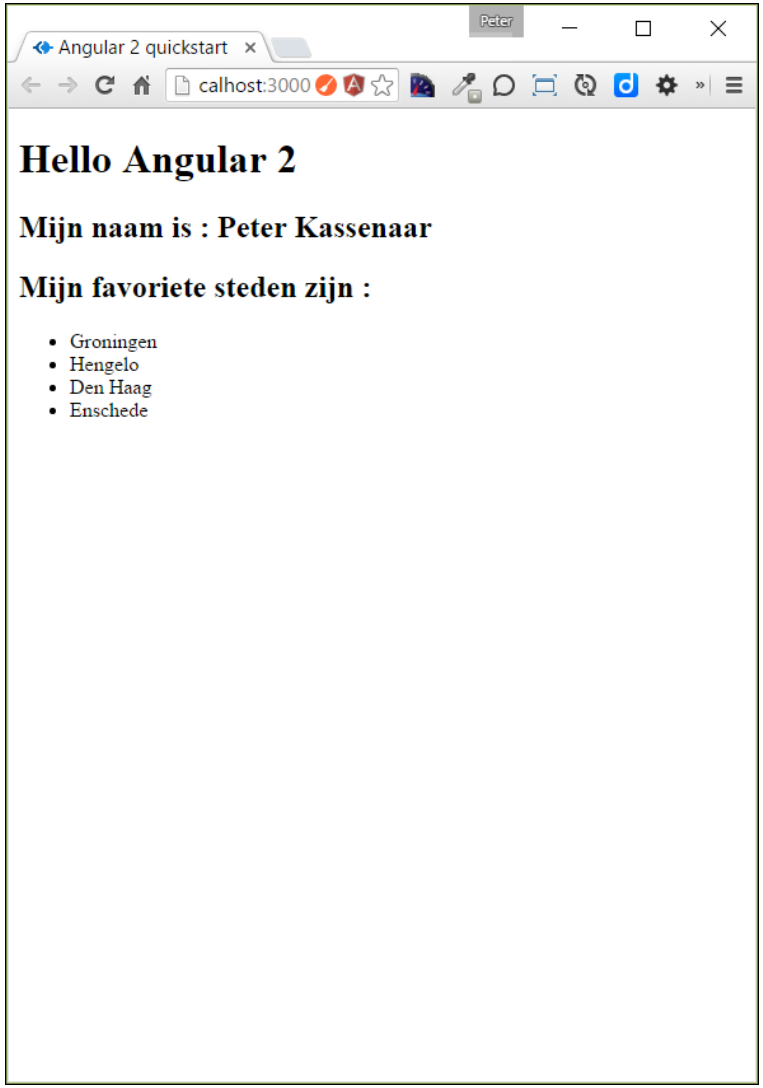
Binden via een lus: `*ngFor`

Template:

```
<h2>Mijn favoriete steden  zijn :</h2>
<ul>
   <li *ngFor="let city of cities">{{ city }}</li> // vanaf .rc1
</ul>
```

Class:

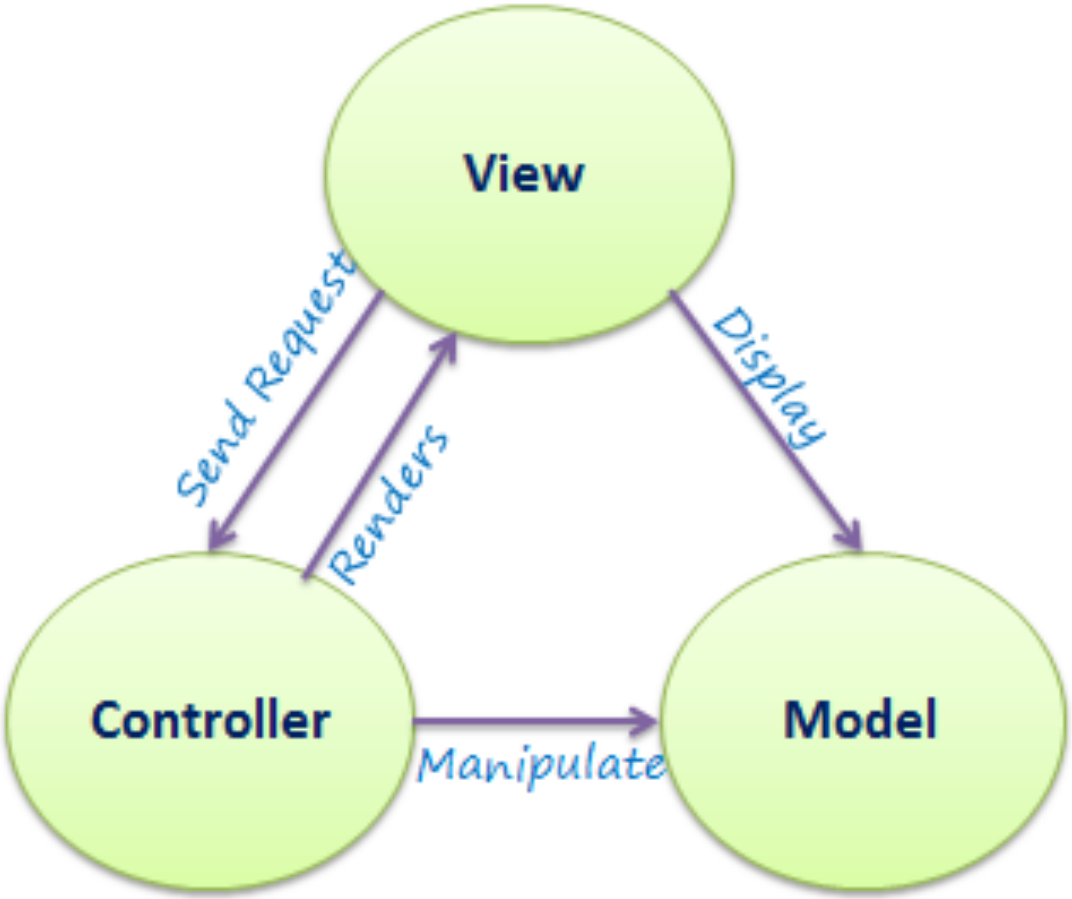
```
// Class met properties, array met cities
export class AppComponent {
   name:string;
   cities:string[];

   constructor() {
      this.name   = 'Peter Kassenaar';
      this.cities = ['Groningen', 'Hengelo', 'Den Haag', 'Enschede']
   }
}
```



Meer info:

https://angular.io/docs/ts/latest/guide/displaying-data.html

MVC – Model, View, Controller



Model maken (als in: MVC)

Class met properties die wordt geëxporteerd:

```
export class City{
   constructor(
      public id: number,
      public name: string,
      public province: string,
   ){ }
}
```

Let op de shorthand notatie bij `public id : number` :

1. Maakt lokale parameter
2. Maakt publieke parameter met zelfde naam
3. Initaliseert parameter bij instantiering van de class met `new`

Model gebruiken

1. Model-class importeren

```
import {City} from './city.model'
```

2. Component aanpassen

```
export class AppComponent {
      name   = 'Peter Kassenaar';
      cities =[
         new City(1, 'Groningen', 'Groningen'),
         new City(2, 'Hengelo', 'Overijssel'),
         new City(3, 'Den Haag', 'Zuid-Holland'),
         new City(4, 'Enschede', 'Overijssel'),
      ]
}
```

3. View aanpassen

```
<li *ngFor="let city of cities">{{ city.id}} - {{ city.name }}</li>
```

Voorwaardelijk tonen met `*ngIf`

Gebruik de directive `*ngIf` (let op het sterretje!)

```
<h2 *ngIf="cities.length > 3">Jij hebt veel favoriete steden!</h2>
```

Externe templates

Als je niet van inline HTML houdt:

```
@Component({
   selector   : 'hello-world',
   templateUrl: './app.html'
})
```

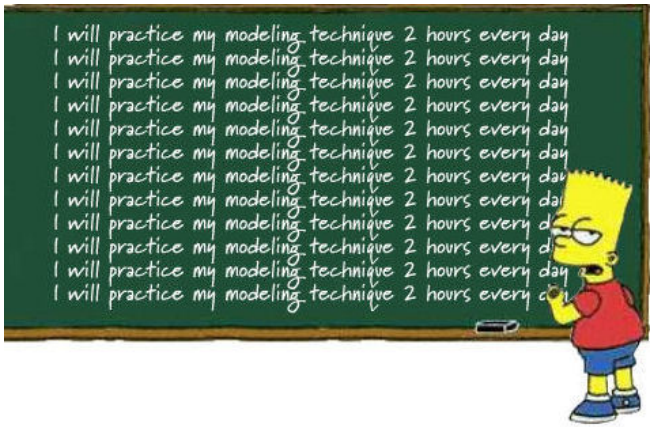
Bestand `app.html`

```
<!-- HTML in externe template -->
<h1>Hello Angular 2</h1>
<p>Dit is een externe template</p>
<h2>Mijn naam is : {{ name }}</h2>
<h2>Mijn favoriete steden  zijn :</h2>
…
```

Checkpoint

- Simple data binding `{{ … }}`
- Properties van de class worden gebonden
- Lussen en voorwaardelijke statement via `*ngFor` en `*ngIf`
- Eventueel externe HTML-templates

Oefening….



User input en event binding

Reageren op mouse, keyboard, hyperlinks en meer

Event binding syntaxis

Gebruik ronde haken voor events:

Angular 1:

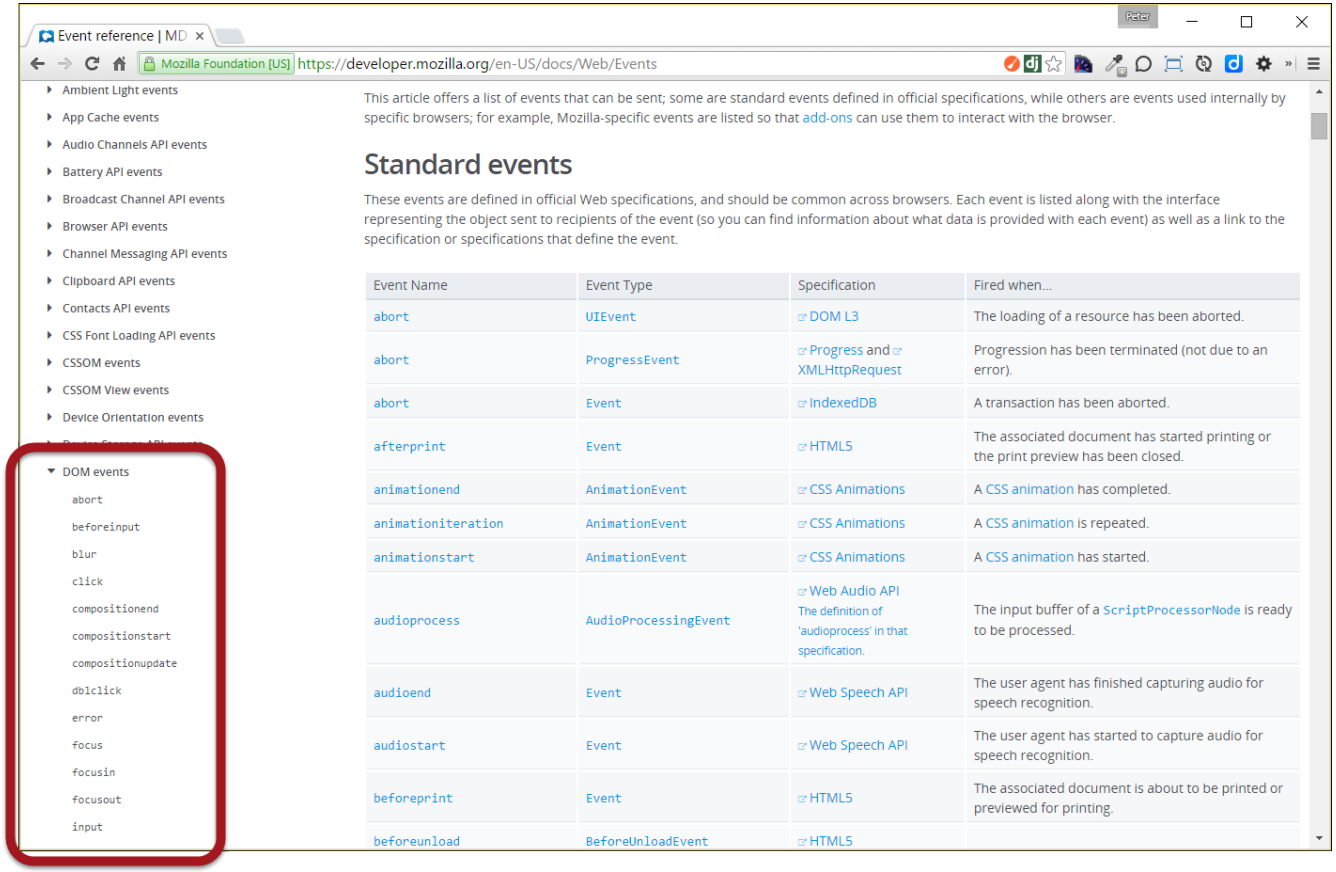
```
<div ng-click="handleClick()">…</div>
```

Angular 2:

```
<div (click)="handleClick()">…</div>

<div (blur)="onBlur()">…</div>
```

DOM-events

Angular2 kan naar *elk* DOM-event luisteren, zonder dat er een aparte directive voor nodig is:



https://developer.mozilla.org/en-US/docs/Web/Events

Voorbeeld event binding

HTML

```
<!-- Event binding voor een button -->
<button class="btn btn-success"
      (click)="btnClick()">Ik ben een button</button>
```

Class

```
export class AppComponent {
   …
   counter: number =0;

   btnClick(){
      alert('Je hebt '+ ++this.counter +' keer geklikt');
   }
 }
```

- Veel editors geven intellisense voor de beschikbare events
- In Angular taal:
  - Links van het isgelijkteken: ***target of the binding***
  - Rechts van het isgelijkteken: ***template expression***
  - Template expression wordt uitgevoerd in de ***execution context*** (= huidige class)

Event binding met $event

HTML

```
<input type="text" class="input-lg" placeholder="Plaatsnaam..."
      (keyup)="onKeyUp($event)"><br>
<p>{{ txtKeyUp}}</p>
```

Class

```
// 2. Binden aan keyUp-event in de textbox
onKeyUp(event:any){
   this.txtKeyUp = event.target.value + ' - ';
}
```

Probleem: `event` is niet strongly typed. Als je dat echter wel doet, wordt de class veel minder portable

Oplossing: gebruik **local template variable** (zeg maar een soort “id” voor het element)

Binding met local template variable

Declareer *local template variable* met # → Het hele element wordt doorgegeven aan de component

Let op: wél binden aan event, anders gebeurt er niks.

```
<input type="text" class="input-lg" placeholder="Plaatsnaam..."
      #txtCity (keyup)="betterKeyUp()">
<h3>{{ txtCity.value }}</h3>
```

Class:

```
 // 3. Binden aan keyUp-event via local template variable
betterKeyUp(){
   //... do nothing, for now
}
```

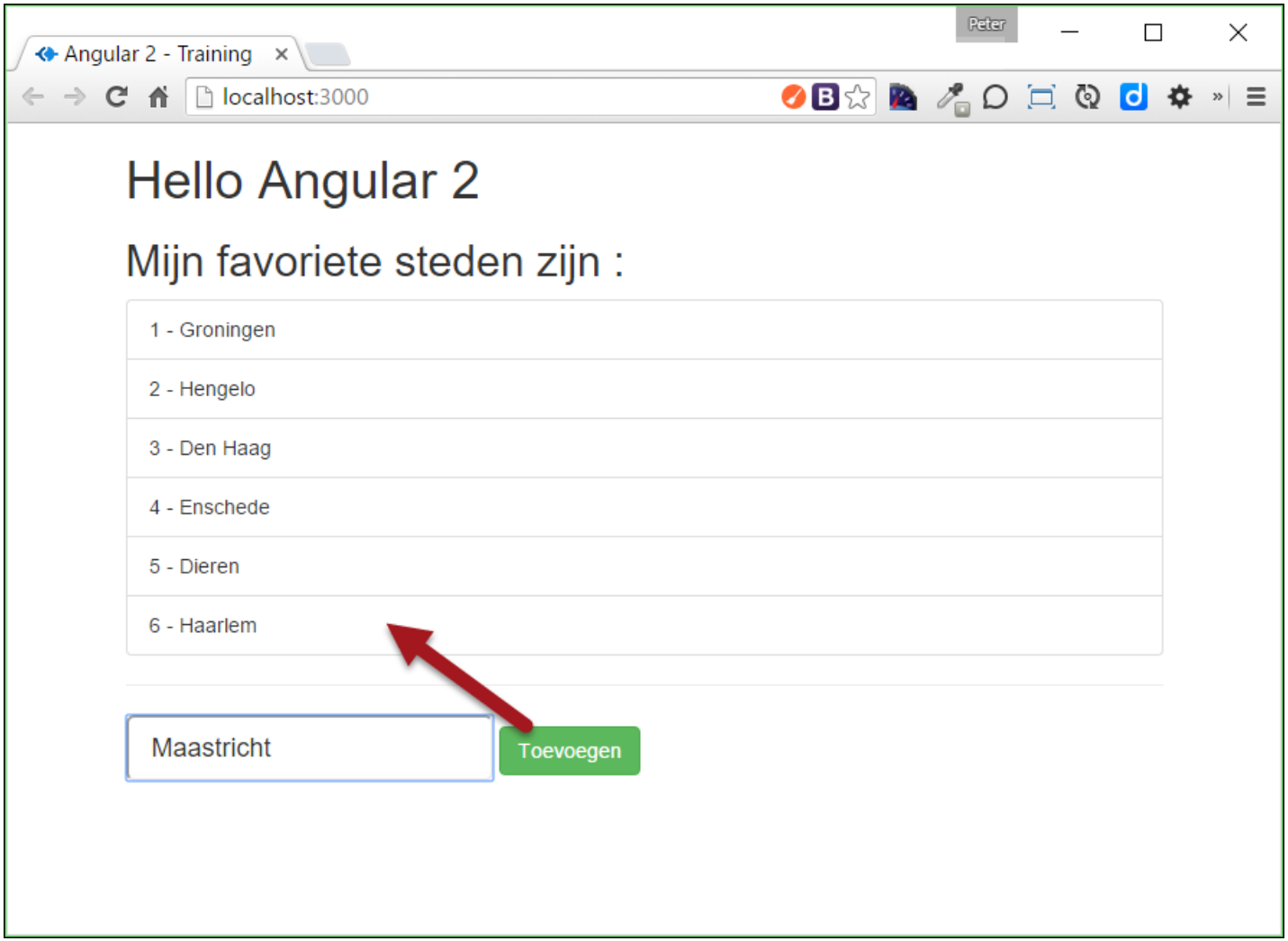
Putting it all together…

HTML

```
<input type="text" class="input-lg" placeholder="Plaatsnaam..." #txtCity>
<button class="btn btn-success"
      (click)="addCity(txtCity)">Toevoegen
</button>
```

Class

```
export class AppComponent {
   // Properties voor de component/class
   …
   addCity(txtCity) {
      let newID   = this.cities.length + 1;
      let newCity = new City(newID, txtCity.value, 'Onbekend');
      this.cities.push(newCity);
      txtCity.value = '';
   }
}
```

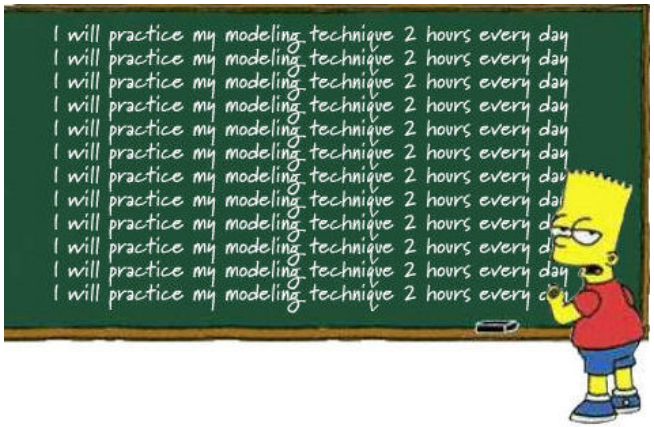


Verder lezen/meer informatie: https://angular.io/docs/ts/latest/guide/user-input.html

Checkpoint

- Event binding wordt aangegeven met `(eventName)="…"`
- Events worden afgehandeld door een event handler-functie in de component
- Op deze manier zijn eenvoudige CRUD-operations te realiseren.

Oefening….



Attribute & property binding

Eigenschappen binden aan HTML-attributen en DOM-properties

Attribute binding syntaxis

Rechtstreeks binden aan properties van HTML-elementen.
Ook wel: *one-way binding.*
Gebruik blokhaken syntaxis

Angular 1:

```
<div ng-hide="true|false">…</div>
```

Angular 2:

```
<div [hidden]="true">…</div>
```

Of :

```
<div [hidden]="person.hasEmail">…</div>
<div [style.background-color]="'yellow'">…</div>
```

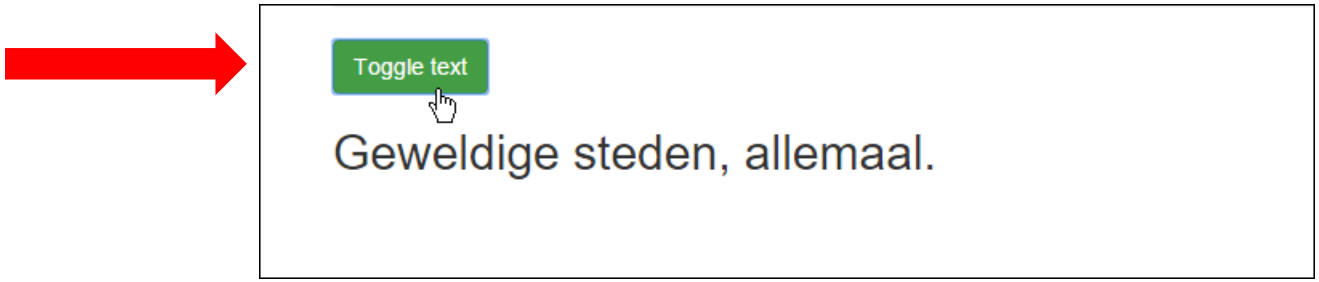
Voorbeeld attribute binding

HTML

```
<!-- Attribute binding -->
<button class="btn btn-success" (click)="toggleText()">Toggle text</button>
<h2 [hidden]="textVisible">Geweldige steden, allemaal.</h2>
```

Class

```
// attribuut toggelen: tekst zichtbaar/onzichtbaar maken.
toggleText(){
   this.textVisible = !this.textVisible;
}
```



Bijvoorbeeld…

HTML

```
<li *ngFor="#city of cities" class="list-group-item"
   (click)="updateCity(city)">
   {{ city.id}} - {{ city.name }}
</li>
```

Class

```
export class AppComponent {
   // …
   currentCity:City    = null;
   cityPhoto:string    = '';

   // Geselecteerde city updaten in de ui. Nieuw : ES6 String interpolation
   updateCity(city:City) {
      this.currentCity = city;
      this.cityPhoto   = `img/${this.currentCity.name}.jpg`;
   }
}
```

Demo:

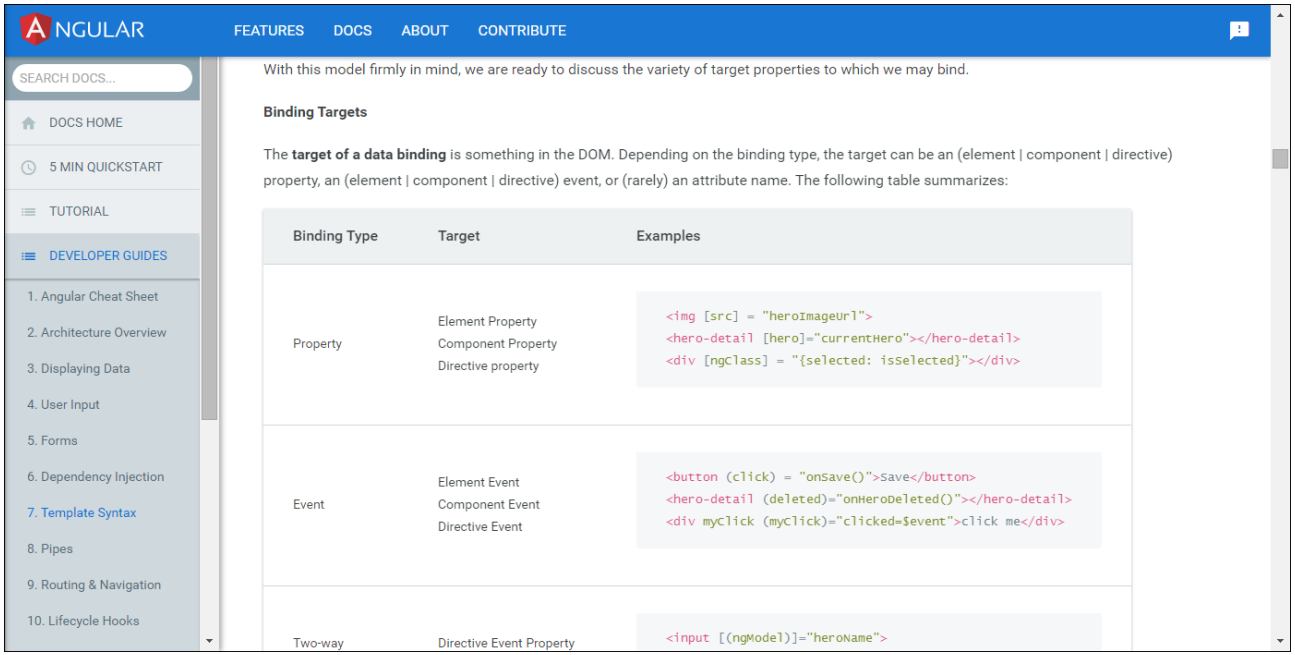
`..\04-attributebinding\app\app-02.html` en

`..\app-02.component.ts`

Meer informatie: https://angular.io/docs/ts/latest/guide/template-syntax.html#!#property-binding

Meer binding-opties

- Attribute binding en DOM-property binding
- Class binding
- Style binding
- https://angular.io/docs/ts/latest/guide/template-syntax.html



Two-way binding

User interface en logica gelijktijdig updaten

Two way binding syntaxis

Is een tijdje weg geweest uit Angular 2, maar op veler verzoek toch teruggekeerd

Angular 1:

```
<input ng-model="person.firstName" />
```

Angular 2: de notatie is een beetje bizar:

```
<input [(ngModel)]="person.firstName" />
```

[(ngModel)] gebruiken

HTML

```
<input type="text" class="input-lg" [(ngModel)]="newCity" />
<h2>{{ newCity }}</h2>
```

Dat is shorthand-notatie voor:

```
<!-- Two-way binding met uitgebreide syntaxis-->
<input type="text" class="input-lg"
      [value]="newCityExtended"
      (input)="newCityExtended = $event.target.value" />
<h2>{{ newCityExtended }}</h2>
```

FormsModule importeren

- Vroeger maakte de Formulier-functionaliteit standaard deel uit van Angular.
- Nu niet meer – apart importeren in `app.module.ts`!

- `import {FormsModule} from "@angular/forms";`
- …
- `imports  : [BrowserModule, FormsModule],`

Binding cheat sheet

ANGULAR FEATURES DOCS ABOUT CONTRIBUTE

SEARCH DOCS...

DOCS HOME

5 MIN QUICKSTART

TUTORIAL

DEVELOPER GUIDES

1. Angular Cheat Sheet
2. Architecture Overview
3. Displaying Data
4. User Input
5. Forms
6. Dependency Injection
7. Template Syntax
8. Pipes
9. Routing & Navigation
10. Lifecycle Hooks
11. Attribute Directives
12. Structural Directives
13. Hierarchical Injectors

ANGULAR CHEAT SHEET

Angular 2 for TypeScript

This cheat sheet is provisional and may change. Angular 2 is currently in Beta.

Angular for TypeScript Cheat Sheet (v2.0.0-beta.0)

Bootstrapping	`import {bootstrap} from 'angular2/angular2';`
`bootstrap(MyAppComponent, [MyService, provide(...)]);`	Bootstraps an application with MyAppComponent as the root component and configures the DI providers.

Template syntax	
`<input [value]="firstName">`	Binds property `value` to the result of expression `firstName`.
`<div [attr.role]="myAriaRole">`	Binds attribute `role` to the result of expression `myAriaRole`.
`<div [class.extra-sparkle]="isDelightful">`	Binds the presence of the CSS class `extra-sparkle` on

https://angular.io/docs/ts/latest/guide/cheatsheet.html

Ingebouwde directives

Veel directives konden vervallen door de nieuwe syntaxis. Er zijn er nog maar weinig over.

Directives die het DOM manipuleren: herkenbaar aan sterretje/asterisk

```
<div *ngFor="person of Persons">…</div>
<div *ngIf="showDiv">…</div>
<div [ngClass]="setClasses()">…</div>
<div [ngStyle]="setStyles()">…</div>
```

Samenvatting…

- Databinding is in Angular 2 vernieuwd
- Leer werken met de nieuwe notatie voor DOM- en Attribute binding, event binding en two-way binding
- Pas altijd de Component en de bijbehorende View aan.
- Veel concepten komen overeen, de uitwerking is totaal nieuw, in vergelijking met Angular 1